

TRABALHO 2 — SISTEMA WEB COM CRUD

Flask + Python + SQL + HTML + CSS

Objetivo do Trabalho

Este trabalho tem como objetivo consolidar os conceitos de modelagem e manipulação de bancos de dados relacionais, integrados a uma aplicação web funcional. Cada aluno (ou dupla) deverá escolher um tema, modelar o banco de dados correspondente e desenvolver um sistema web completo em Python com o framework Flask, capaz de Cadastrar, Consultar, Editar e Excluir registros (CRUD completo).

Além da parte de banco de dados e back-end, o sistema também será avaliado quanto à acessibilidade da interface — ou seja, o quanto a página é utilizável por pessoas com diferentes necessidades (baixa visão, uso apenas do teclado, entre outras).

Escolha do Tema do Banco de Dados

O aluno ou a dupla deve escolher livremente um tema para o sistema, desde que ele permita uma modelagem de banco de dados com pelo menos duas tabelas relacionadas. Alguns exemplos de tema (não obrigatórios, apenas sugestões):

Temas mais "clássicos"

- Livraria (livros, autores, empréstimos)
- Clínica veterinária (pets, tutores, consultas)
- Escola / cursos (alunos, turmas, matrículas)
- Loja de roupas / e-commerce (produtos, categorias, pedidos)
- Academia (alunos, planos, treinos)
- Locadora de filmes / games (títulos, categorias, aluguéis)
- Delivery de comida (cardápio, clientes, pedidos)
- Biblioteca (acervo, usuários, empréstimos)
- Controle de estoque (produtos, fornecedores, entradas/saídas)

Temas de Jogos (games)

- Biblioteca de jogos pessoal (jogos, plataformas/consoles, horas jogadas, status de conclusão).
- Clube de e-sports (jogadores, times, campeonatos, partidas).
- Loja de skins/itens de jogo (itens, raridades, jogadores, inventário).
- Sistema de torneios de videogame (competidores, partidas, fases, classificação).
- Guild / clã de um jogo online (membros, cargos, eventos, conquistas).
- Ranking de speedrun ou de recordes em jogos (jogadores, jogos, tempos registrados).
- Locadora ou marketplace de jogos usados (jogos, vendedores, compradores, avaliações).

Temas de Redes Sociais e Conteúdo Digital

- Mini rede social (usuários, posts, curtidas/comentários).
- Plataforma de streaming de séries/filmes (usuários, títulos, avaliações, listas "favoritos").
- Rede social de fãs de um artista/banda (usuários, posts, playlists).
- Sistema de fórum de discussão por tema (usuários, tópicos, respostas).
- Plataforma de criadores de conteúdo (criadores, vídeos/posts, seguidores, categorias).
- Grupo de estudos colaborativo (usuários, matérias, materiais compartilhados, comentários).
- App de playlists de música (usuários, músicas, playlists, artistas).

O tema deve ser justificado no relatório: por que foi escolhido e o que o sistema resolve na prática. Fiquem à vontade para escolher (ou adaptar) o tema que mais fizer sentido para vocês — o importante é que ele permita pelo menos duas tabelas relacionadas com um CRUD completo.

Modelagem do Banco de Dados

Antes de programar, o aluno deve modelar o banco de dados do tema escolhido. A modelagem deve conter, no mínimo:

- Pelo menos 2 entidades (tabelas) relacionadas entre si (ex.: Cliente e Pedido; Aluno e Matrícula).
- Para cada tabela: nome, lista de atributos (colunas) e seus tipos de dado (INTEGER, TEXT, REAL, DATE etc.).
- Identificação da chave primária (PRIMARY KEY) de cada tabela.
- Identificação de pelo menos uma chave estrangeira (FOREIGN KEY), representando o relacionamento entre as tabelas.
- O tipo de relacionamento entre as tabelas (um-para-muitos ou muitos-para-muitos).

A modelagem deve ser representada no relatório por um diagrama entidade-relacionamento (DER/MER), podendo ser feito à mão (foto/scan) ou em ferramentas gratuitas como draw.io, dbdiagram.io ou Lucidchart.

Tecnologias Obrigatórias

Python + Flask: framework para criar as rotas do sistema (páginas e ações de cadastrar, consultar, editar, excluir).

Banco de Dados SQL: recomenda-se o SQLite (biblioteca sqlite3, já incluída no Python) por ser simples de configurar; outros bancos (MySQL, PostgreSQL) também são aceitos, desde que o aluno se responsabilize pela configuração.

HTML: estrutura das páginas, usando o motor de templates do Flask (Jinja2).

CSS: estilização das páginas, em um ou mais arquivos externos (não usar apenas estilo inline).

Sugestão de estrutura de pastas do projeto:

```
meu_projeto/
├── app.py           (rotas Flask e conexão com o banco)
├── banco.db         (arquivo do banco SQLite)
├── templates/
│   ├── index.html  (página principal)
│   ├── cadastrar.html (formulário de cadastro)
│   └── consultar.html (listagem dos registros)
```

```

|   └─ editar.html      (formulário de edição)
└─ static/
    └─ css/
        └─ style.css    (estilos das páginas)

```

Funcionalidades Obrigatórias (CRUD Completo)

O sistema deve conter, no mínimo, as seguintes páginas e funcionalidades:

Página / Funcionalidade	O que deve fazer
Página Principal (Home)	Tela inicial do sistema, com o nome/logo do projeto, um breve texto de apresentação do tema escolhido e links/menu de navegação para as demais páginas (Cadastrar, Consultar).
Formulário de Cadastro (Create)	Formulário HTML com campos correspondentes aos atributos da tabela principal. Ao enviar, os dados devem ser salvos no banco via uma rota Flask (INSERT).
Página de Consulta / Listagem (Read)	Lista (tabela HTML) com todos os registros cadastrados, buscando os dados do banco (SELECT). Recomenda-se incluir um campo de busca/filtro simples.
Edição (Update)	A partir da listagem, o usuário pode clicar em "Editar" em um registro específico, abrir um formulário pré-preenchido com os dados atuais e salvar as alterações (UPDATE).
Exclusão (Delete)	A partir da listagem, o usuário pode remover um registro (DELETE), com uma confirmação antes de excluir (ex.: janela de confirmação ou página de confirmação).

Todas as páginas devem estar visualmente integradas (mesmo cabeçalho/menu de navegação, mesma paleta de cores e fontes), como se fossem partes de um único sistema.

Regras de Acessibilidade

Acessibilidade é a prática de projetar interfaces que possam ser usadas pelo maior número possível de pessoas, incluindo pessoas com baixa visão, daltonismo, dificuldades motoras (que usam apenas o teclado) ou outras limitações. O sistema desenvolvido deve seguir as regras abaixo, baseadas nas diretrizes WCAG (Web Content Accessibility Guidelines). Para cada regra, a tabela indica também como verificar se ela foi cumprida.

Regra	O que fazer	Como verificar
Contraste de cores	Texto normal deve ter contraste mínimo de 4,5:1 com o fundo; textos grandes (a partir de 18pt ou 14pt em negrito) podem usar 3:1. Evite combinações como texto cinza claro sobre fundo branco.	Testar cada par de cores (texto/fundo) no WebAIM Contrast Checker (webaim.org/resources/contrastchecker) ou usar a aba "Accessibility" do Lighthouse no Chrome DevTools.
Rótulos em formulários	Todo campo de formulário (<input>, <select>, <textarea>) deve ter uma <label> associada, usando o atributo for ligado ao id do campo.	Clicar sobre cada label e verificar se o foco vai para o campo correspondente; ou rodar a extensão axe DevTools e conferir se não aparece o erro "Form elements must have labels".
Navegação por teclado	Toda a página deve poder ser utilizada apenas com o teclado (Tab, Shift+Tab, Enter), em uma ordem lógica, com o item em foco sempre	Desconectar o mouse (ou simplesmente não usá-lo) e navegar pela página inteira só com

Regra	O que fazer	Como verificar
	visível (não usar <code>outline:none</code> sem substituir por outro indicador visual).	Tab/Enter, conferindo se dá para preencher o formulário e usar os botões normalmente.
Estrutura semântica do HTML	Usar tags semânticas (<code><header></code> , <code><nav></code> , <code><main></code> , <code><footer></code> , <code><form></code> , <code><table></code>) em vez de apenas <code><div></code> para tudo; manter uma hierarquia correta de títulos (h1, depois h2, depois h3, sem pular níveis).	Inspecionar o HTML pelo DevTools (aba Elements) e conferir a árvore de tags e a ordem dos títulos; rodar o Lighthouse (categoria Acessibilidade) e verificar a pontuação.
Texto alternativo em imagens	Toda imagem com conteúdo relevante deve ter o atributo alt descrevendo-a; imagens puramente decorativas devem usar <code>alt=""</code> .	Inspecionar cada <code></code> no HTML e conferir se o atributo alt está presente e faz sentido com o conteúdo da imagem.
Tamanho da área clicável	Botões e links devem ter uma área de clique de pelo menos 44x44 pixels, para facilitar o uso em telas de toque.	Selecionar o botão/link no DevTools e verificar a largura e altura da caixa (box model) no painel de estilos.
Não depender só da cor	Mensagens de erro/sucesso não podem ser indicadas apenas pela cor (ex.: só a borda vermelha); deve haver também um texto ou ícone explicando a mensagem.	Revisar visualmente cada mensagem de validação do formulário e confirmar que existe um texto explicativo, além da cor.
Responsividade básica	O layout deve continuar legível e utilizável em telas menores, sem cortar conteúdo nem exigir rolagem horizontal.	Usar o modo "Device Toolbar" do Chrome DevTools e testar a página em uma largura de aproximadamente 360-400px.

No relatório, o aluno deve descrever, para cada uma dessas regras, como ela foi aplicada no seu projeto e anexar evidências da verificação (ex.: print do resultado do Lighthouse, print do WebAIM Contrast Checker com os códigos de cor usados, ou uma breve descrição do teste de navegação por teclado).

Ferramenta recomendada para escolher a paleta de cores

Para facilitar a escolha de cores que já nascem com bom contraste, use o Adobe Color (color.adobe.com). Nele é possível montar uma paleta harmônica a partir de uma cor principal, testar combinações prontas (na seção "Explorar") e até verificar simulações de daltonismo. Depois de escolher a paleta no Adobe Color, confirme o contraste de cada combinação texto/fundo no WebAIM Contrast Checker antes de aplicar no CSS.

Entrega do Trabalho

A entrega deve conter os arquivos e informações abaixo:

1. Código-fonte completo do sistema: arquivo `app.py`, pasta `templates/`, pasta `static/` (com o CSS) e o arquivo do banco de dados (ou o script SQL de criação das tabelas), tudo funcionando.

2. Relatório (Word / Google Docs / PDF), contendo:

- Título do projeto e tema escolhido, com justificativa.
- Modelagem do banco de dados: diagrama entidade-relacionamento (DER/MER), lista de tabelas, atributos, tipos, chaves primárias e estrangeiras.
- Descrição das funcionalidades implementadas (Cadastrar, Consultar, Editar, Excluir), com prints de tela de cada página.
- Relatório de acessibilidade: quais das regras da tabela acima foram aplicadas e como foram verificadas (com prints/evidências).

Uso de Inteligência Artificial: informar se alguma IA foi utilizada durante o desenvolvimento; em caso afirmativo, qual ferramenta (ex.: ChatGPT, Claude, GitHub Copilot) e como foi empregada (ex.: geração de código, tirar dúvidas, revisão, depuração de erros).

Uso de IA para o design (HTML/CSS): informar especificamente se alguma IA foi utilizada para ajudar na parte visual (layout, paleta de cores, CSS); em caso afirmativo, qual foi a IA utilizada e qual foi o prompt (o texto exato pedido à IA) usado para gerar ou ajustar o design.

- Uma reflexão respondendo: Qual foi a parte mais difícil de fazer neste trabalho?

Critérios de Avaliação (total 100 pontos)

Critério de Avaliação	Pontos	Descrição
Modelagem do Banco de Dados	15	Modelagem coerente com o tema escolhido: entidades bem definidas, atributos, chaves primárias e estrangeiras, e pelo menos um relacionamento entre tabelas.
CRUD Completo (Create, Read, Update, Delete)	30	As quatro operações funcionam corretamente, persistindo e refletindo as alterações no banco de dados.
Integração Flask + Banco de Dados	15	Rotas Flask organizadas, conexão com o banco funcionando corretamente e tratamento de erros básicos (ex.: campos vazios, ids inexistentes).
HTML e CSS (estrutura e estilização)	10	Páginas bem estruturadas, estilização consistente entre as telas e uso adequado de um arquivo CSS externo.
Acessibilidade	15	Aplicação das regras de acessibilidade descritas neste documento (contraste, rótulos, navegação por teclado, estrutura semântica etc.), com evidências no relatório.
Relatório	10	Relatório completo, com modelagem, explicação das funcionalidades, verificação de acessibilidade, uso de IA e reflexão final.
Funcionamento Geral e Usabilidade	5	O sistema roda sem erros, é fácil de navegar e a experiência de uso é consistente entre as páginas.
TOTAL	100	

Dicas Gerais

- Comece pela modelagem: só depois de decidir as tabelas, atributos e relacionamentos, comece a escrever o código — isso evita retrabalho.
- Crie o banco com um script separado (ex.: criar_banco.py) que roda uma única vez, criando as tabelas com CREATE TABLE.
- Use um único arquivo style.css compartilhado entre todas as páginas, para manter a identidade visual consistente.
- Teste o contraste de cores desde o início do desenvolvimento do CSS, não apenas no final — é mais fácil ajustar aos poucos.
- Sempre valide os dados recebidos do formulário no back-end (Flask), além de eventuais validações no HTML (ex.: required), para evitar que o banco receba dados inválidos ou vazios.

- Teste o CRUD completo antes de entregar: cadastre um registro, consulte, edite e exclua, conferindo se o banco realmente reflete cada operação.

Sobre o uso de IA no layout e na acessibilidade: vocês podem usar a IA Claude como apoio para pensar o layout das páginas e revisar as regras de acessibilidade (por exemplo, pedindo sugestões de paleta de cores acessível, perguntando se um trecho de HTML está usando as tags semânticas corretas, ou pedindo dicas de como estruturar os labels do formulário). **Atenção:** não copiem e cole código pronto gerado pela IA direto no projeto, pois ele não vai seguir a estrutura de pastas, os nomes de variáveis/rotas e a lógica que vocês construíram no Flask — o código provavelmente não vai funcionar sem ajustes, ou vai quebrar a integração com o banco. Usem a IA para tirar dúvidas e obter sugestões, mas escrevam e adaptem o código vocês mesmos, entendendo cada parte.